PolyFlop8

Block Specification Manual

Component Interfaces & Internal Logic

Revision: 1.3
Date: January 6, 2026

Generated for Test Bench Development

1 REVISION HISTORY

Revision	Change	Date
1.0	Initial document creation	04.01.2026
1.1	Change general architecture. Add ALU in to DRAM addr mux.	05.01.2026
1.2	Change RegFile addr_b to accept 8-bit and slice bus internally due to architectural decision	06.01.2026
1.3	Verify functionality against actual implementation and fix indescrepancies	06.01.2026

2 POLYFLOP8 GENERAL ARCHITECTURE

The PolyFlop8 is a simple 8-bit microcontroller implemented in synthesizable VHDL for FPGA targets. Maybe it has a 1970s vibe, but at least it's also unoptimized and sluggish.

The processor follows a strict **Harvard Architecture** with separate Program ROM and Data RAM. The simple 2-stage pipeline allows simultaneous instruction fetch and execution.

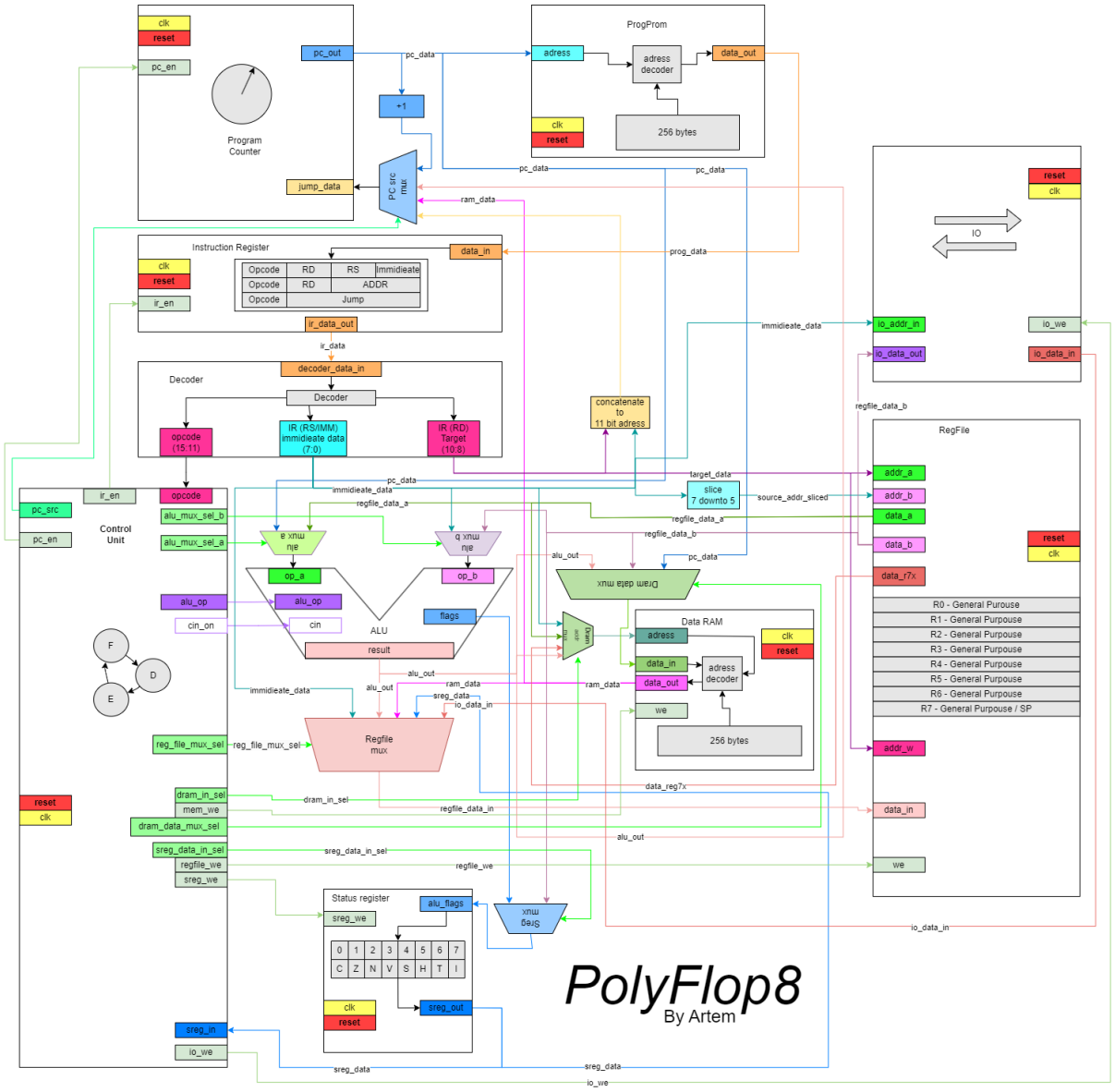

Figure 1: PolyFlop8 processor

3 ALU (ARITHMETIC LOGIC UNIT)

The ALU is a combinational block responsible for arithmetic and logical operations. It features internal multiplexers for operand selection and a flag generation unit. The design is validated under test case `TC-ALU-001`.

3.1 Implementation Schematic

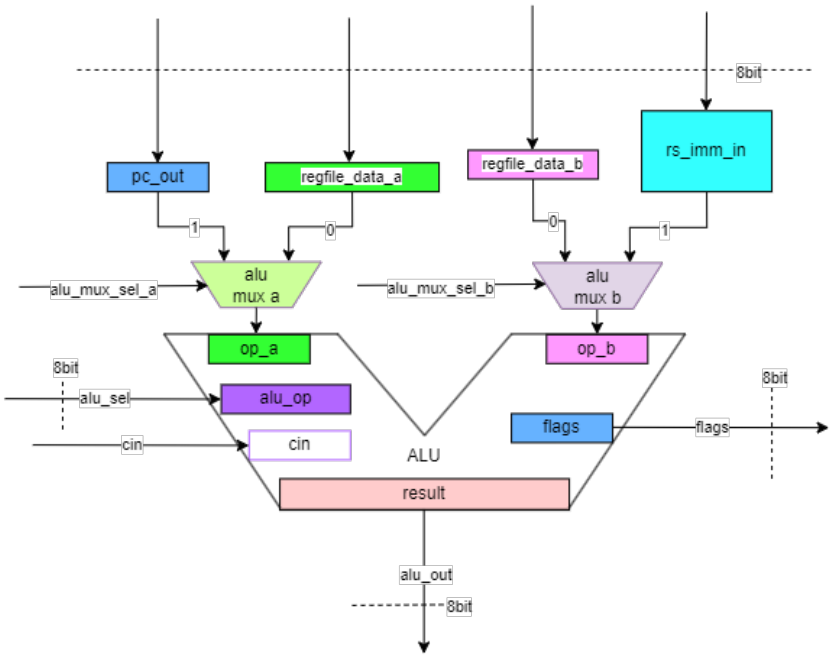

Figure 2: ALU High-Level Architecture

3.2 Interface Signals

Signal Name	Dir	Width	Description
`pc_data`	IN	8-bit	Program Counter value (Channel A alternate input)
`regfile_data_a`	IN	8-bit	Register File Port A (Channel A default input)
`rs_imm_in`	IN	8-bit	Immediate value or Source Register (Channel B alternate input)
`regfile_data_b`	IN	8-bit	Register File Port B (Channel B default input)
`alu_mux_sel_a`	IN	1-bit	Mux A Select: ’0’ = RegFile A, ’1’ = PC
`alu_mux_sel_b`	IN	1-bit	Mux B Select: ’0’ = RegFile B, ’1’ = Imm/RS
`alu_sel`	IN	4-bit	Operation Opcode (See Table 3.3)
`cin`	IN	1-bit	Carry Input (used for ADC, SBC)
`result`	OUT	8-bit	Result of the operation
`flags`	OUT	5-bit	Status Flags: H(4), V(3), N(2), C(1), Z(0)

Table 1: ALU Entity Port Definition

3.3 Operations Table (`alu_sel`)

The ALU operations correspond to the VHDL constants defined in the behavioral architecture.

Code	Name	Operation	Flags Affected
0000	ADD	$Result = A + B$	H, V, C, N, Z
0001	ADC	$Result = A + B + C_{in}$	H, V, C, N, Z
0010	SUB	$Result = A - B$	V, C, N, Z
0011	SBC	$Result = A - B - C_{in}$	V, C, N, Z
0100	AND	$Result = A \wedge B$	Z, N
0101	OR	$Result = A \vee B$	Z, N
0110	XOR	$Result = A \oplus B$	Z, N
0111	MOV	$Result = B$ (Pass-through B)	None
1000	LDI	$Result = A$ (Pass-through A)	None
1001	COM	$Result = \neg A$ (1's complement)	C=1, Z, N
1010	NEG	$Result = 0 - A$ (2's complement)	C, V, Z, N

3.4 Internal Data Path & Multiplexing

The ALU inputs A and B are selected via two internal 2-to-1 multiplexers before reaching the arithmetic core.

- **Operand A Source:** Controlled by `alu_mux_sel_a`.

$$Op_A = \begin{cases} \texttt{regfile\_data\_a} & \text{if } \texttt{alu\_mux\_sel\_a} =' 0' \\ \texttt{pc\_data} & \text{if } \texttt{alu\_mux\_sel\_a} =' 1' \end{cases} \quad (1)$$

- **Operand B Source:** Controlled by `alu_mux_sel_b`.

$$Op_B = \begin{cases} \texttt{regfile\_data\_b} & \text{if } \texttt{alu\_mux\_sel\_b} =' 0' \\ \texttt{rs\_imm\_in} & \text{if } \texttt{alu\_mux\_sel\_b} =' 1' \end{cases} \quad (2)$$

3.5 Flag Generation Logic

Flags are calculated based on the 9-bit internal result (8-bit data + 1-bit carry/overflow) of the operation.

- **Z (Zero Flag):** Set if the 8-bit result is strictly $0x00$.
- **N (Negative Flag):** Set to the Most Significant Bit (MSB, bit 7) of the result.
- **C (Carry Flag):**
  - For ADD/ADC: Set if the result exceeds 8 bits ($Result[8] = 1$).
  - For SUB/SBC: Set if a borrow occurs (derived from internal bit 8 during subtraction).
  - For COM: Always set to '1'.
  - For NEG: Set to '1' if $Result \neq 0$, else '0'.
- **V (Overflow Flag):** Indicates signed overflow using 2's complement logic.

$$V_{ADD} = (A_7 \cdot B_7 \cdot \overline{R_7}) + (\overline{A_7} \cdot \overline{B_7} \cdot R_7) \tag{3}$$

- **H (Half Carry Flag):** Used for BCD arithmetic, indicates carry from bit 3 to bit 4.

$$H = (A_3 \cdot B_3) + (B_3 \cdot \overline{R_3}) + (A_3 \cdot \overline{R_3}) \tag{4}$$

4 INSTRUCTION DECODER

The Decoder is a purely combinational block that slices the 16-bit instruction word into usable control signals and immediate values. It requires no clock signal and operates continuously on the `decoder_data_in` bus.

4.1 Implementation Schematic

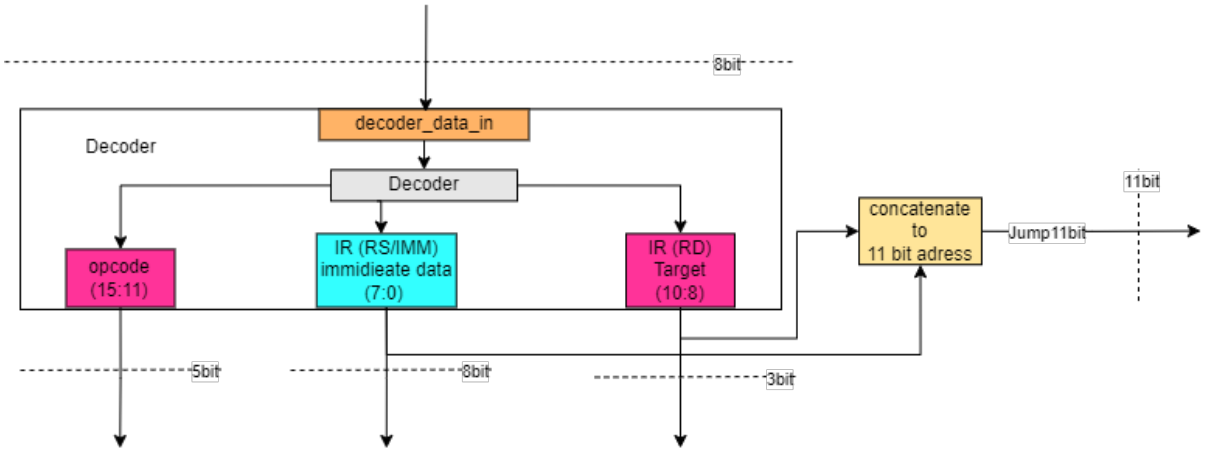

Figure 3: Decoder High-Level Architecture

4.2 Interface Signals

Signal Name	Dir	Width	Description
`decoder_data_in`	IN	16-bit	Full instruction word fetched from ROM
`opcode`	OUT	5-bit	Instruction Opcode (Bits 15 down to 11)
`RD`	OUT	3-bit	Destination Register Address (Bits 10 down to 8)
`RS_IMM`	OUT	8-bit	Combined field for Source Reg or Immediate (Bits 7 down to 0)
`Jump11bit`	OUT	11-bit	Absolute address for JMP/CALL (Bits 10 down to 0)

Table 2: Decoder Entity Port Definition

4.3 Functional Logic

The decoder performs concurrent bit-slicing of the instruction word (I). The mappings are hardwired as follows:

- **Opcode Extraction:**
$$\texttt{opcode} = I[15:11] \tag{5}$$
Determines the operation type (ALU, Branch, Load/Store).
- **Destination Register (RD):**
$$\texttt{RD} = I[10:8] \tag{6}$$
Used as the write address for the Register File.
- **Source/Immediate (RS_IMM):**
$$\texttt{RS\_IMM} = I[7:0] \tag{7}$$
This field is polymorphic. The Control Unit determines interpretation:
  - **Register Mode:** Uses bits $[2:0]$ as Source Register address (Rs).
  - **Immediate Mode:** Uses full $[7:0]$ as an 8-bit constant.
- **Jump Address (Jump11bit):**
$$\texttt{Jump11bit} = I[10:0] \tag{8}$$
Used for J-Type instructions (JMP, CALL). Note that this field overlaps physically with `RD` and `RS_IMM`.

5 PROGRAM COUNTER (PC)

The PC Unit manages the address of the next instruction. It contains an 11-bit register and a 4-way multiplexer to handle sequential execution, jumps, branches, and returns. The module has been verified under Unit/Block Level testing with a PASS status[cite: 4].

5.1 Implementation Schematic

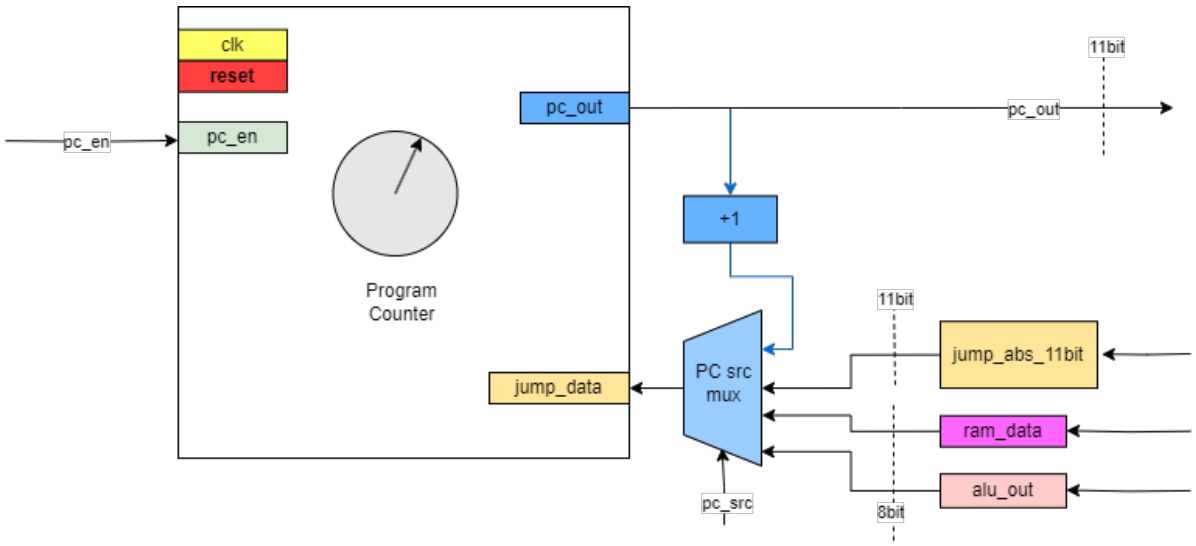

Figure 4: Program Counter

5.2 Interface Signals

Signal Name	Dir	Width	Description
clk	IN	1-bit	System Clock (Tested at 100MHz)
rst	IN	1-bit	Asynchronous Active High Reset
pc_en	IN	1-bit	PC Write Enable (Active High); Stall when Low [cite: 85]
pc_src	IN	2-bit	Source Selector (See Table 5.3)
alu_out	IN	8-bit	Signed offset for Relative Branching
ram_data	IN	8-bit	Return address lower byte (Indirect/RET)
jump_abs_11bit	IN	11-bit	Target address for Absolute Jumps
pc_out	OUT	11-bit	Current Program Counter Address

Table 3: ProgramCounter Entity Port Definition

5.3 Next Address Logic (pc_src)

The next address logic relies on the standard fetch cycle $PC + 1$ combined with the pc_src multiplexer.

pc_src	Mode	Logic Description
00	**Fetch**	$PC \leftarrow PC + 1$ (Sequential execution) [cite: 89]
01	**Branch**	$PC \leftarrow (PC + 1) +$ sign_extend(alu_out) Performs relative jump using signed 8-bit offset added to the next sequential address[cite: 92].
10	**Indirect/Ret**	$PC \leftarrow$ resize(ram_data) Loads address from Memory/Stack. Note: Currently supports page 0 (8-bit) only; upper 3 bits are zeroed.
11	**Jump**	$PC \leftarrow jump\_abs\_11bit$ Loads absolute 11-bit address from instruction[cite: 97].

5.4 Control Flow Priority

The Program Counter operates based on the following priority hierarchy as defined in the behavioral architecture:

1. **Asynchronous Reset:** If rst = '1', the PC is immediately forced to address 0 $(0x000)$, regardless of the clock edge[cite: 82, 83].
2. **Stall/Enable:** On the rising edge of clk, the PC only updates if pc_en = '1'. If pc_en = '0', the PC holds its current value (Stall)[cite: 84, 85].
3. **Next Address Selection:** If enabled, the PC updates based on the pc_src table defined above.

6 REGISTER FILE

The Register File contains 8 general-purpose 8-bit registers (R0-R7). It supports two asynchronous read ports and one synchronous write port with multiple input sources. The module has been verified under Unit/Block Level testing (Rev 1.4) with a PASS status.

6.1 Implementation Schematic

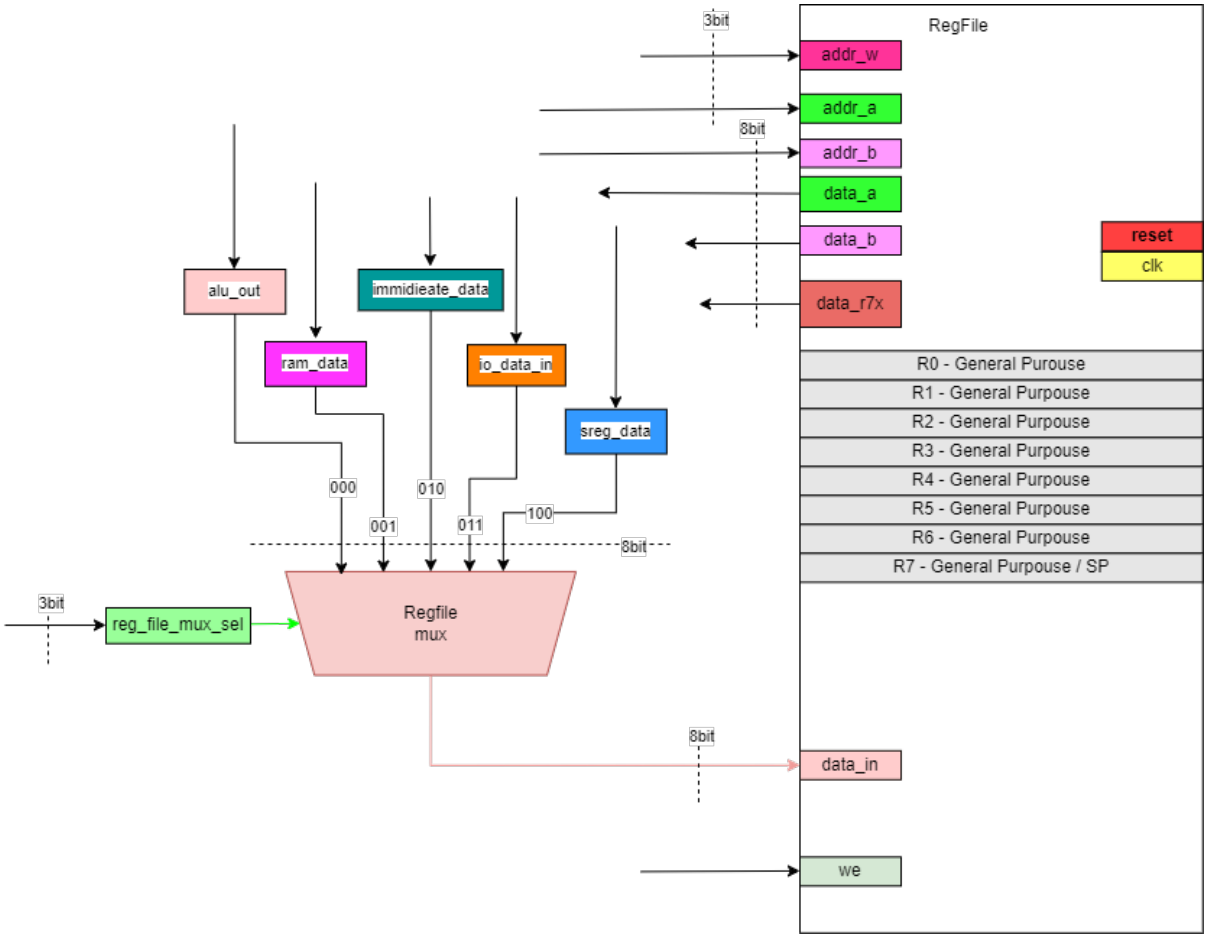

Figure 5: Register File

6.2 Special Functions

- **R7 (X-Pointer):** Register 7 has a dedicated output port `data_r7x`. This allows the Memory Unit to use R7 directly as an address pointer for Indirect Addressing modes without using the ALU.
- **Write Mux:** The `rf_src_sel` signal determines which input bus is written to the destination register when `we` is high.
- **Port B Slicing:** Port B accepts an 8-bit input but uses only the upper 3 bits $(7:5)$ to determine the read address.

6.3 Interface Signals

Signal Name	Dir	Width	Description
clk	IN	1-bit	System Clock
rst	IN	1-bit	Asynchronous Active High Reset
we	IN	1-bit	Write Enable (Active High)
addr_a	IN	3-bit	Read Address Port A
addr_b	IN	8-bit	Read Address Port B (Uses bits 7:5)
addr_w	IN	3-bit	Write Address Port
rf_src_sel	IN	3-bit	Write-Back Source Selector
Write-Back Data Inputs			
alu_out	IN	8-bit	ALU Result
ram_data	IN	8-bit	Data from RAM
rs_imm_in	IN	8-bit	Immediate Value
io_data_in	IN	8-bit	I/O Port Data
sreg_data	IN	8-bit	Status Register Data
Outputs			
data_a	OUT	8-bit	Output Data Port A
data_b	OUT	8-bit	Output Data Port B
data_r7x	OUT	8-bit	Hardwired Output of Register R7 (Index Pointer)

Table 4: RegFile Entity Port Definition

6.4 Write-Back Logic (rf_src_sel)

The write-back multiplexer selects the data source based on the 3-bit rf_src_sel signal.

rf_src_sel	Source Signal	Operation/Instruction
000	alu_out	Arithmetic Ops (ADD, SUB, etc.)
001	ram_data	Memory Load (LD, POP)
010	rs_imm_in	Load Immediate (LDI, MOV)
011	io_data_in	Input from I/O (IN)
100	sreg_data	Status Register Copy
Others	$0x00$	Default Safe State

6.5 Control Flow Priority

The Register File operates based on the following priority hierarchy as defined in the behavioral architecture:

1. **Asynchronous Reset:** If rst = '1', all registers are immediately cleared to $0x00$, ignoring the clock.

2. **Synchronous Write:** On the rising edge of `clk`, if `we = '1'`, the selected data source is written to the register specified by `addr_w`.
3. **Asynchronous Read:** Outputs `data_a`, `data_b`, and `data_r7x` update immediately when address inputs change, regardless of the clock.

7 INSTRUCTION REGISTER (IR)

The Instruction Register acts as a stable buffer between the Program Memory (PROM) and the Decoder. It latches the 16-bit instruction word to ensure signal stability during the decoding and execution phases[cite: 1105, 1171]. The module has been verified under Unit/Block Level testing with a PASS status[cite: 1096].

7.1 Implementation Schematic

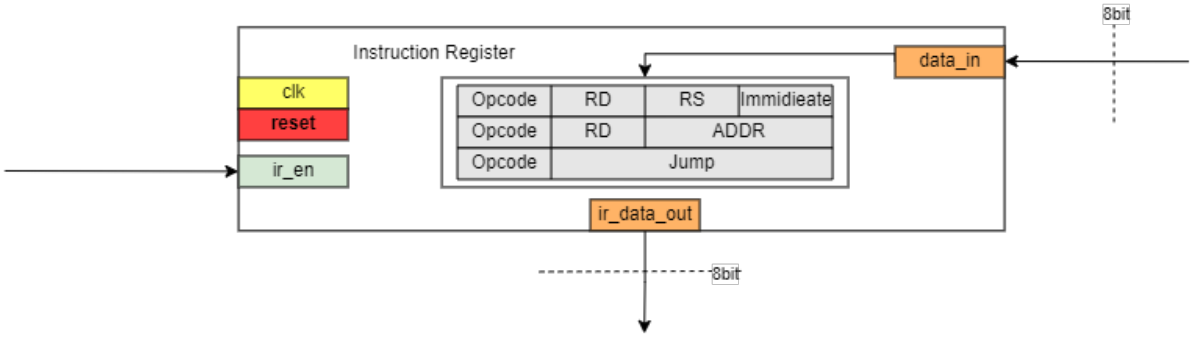

Figure 6: Instruction Register

7.2 Interface Signals

Signal Name	Dir	Width	Description
`clk`	IN	1-bit	System Clock (Tested at 100MHz) [cite: 1101, 1135]
`rst`	IN	1-bit	Asynchronous Active High Reset [cite: 1136, 1155]
`ir_en`	IN	1-bit	Write Enable; active during FETCH state [cite: 1137, 1163]
`data_in`	IN	16-bit	Raw instruction from Program Memory [cite: 1142]
`ir_data_out`	OUT	16-bit	Latched instruction to Decoder/Control Unit [cite: 1144, 1173]

Table 5: Instruction Register Port Definition

7.3 Behavioral Logic

The Instruction Register utilizes a single process sensitive to the clock and reset signals to manage instruction latching.

1. **Asynchronous Reset:** When `rst` is high, the register is immediately cleared to $0x0000$ (representing a NOP instruction) regardless of the clock state[cite: 1156, 1157, 1302].
2. **Synchronous Latching:** On the rising edge of `clk`, the register latches `data_in` only if `ir_en` is asserted[cite: 1161, 1164, 1165].
3. **Data Hold:** If `ir_en` is low, the register maintains its current value, providing a stable signal for the DECODE and EXECUTE cycles[cite: 1169, 1171, 1282].

7.4 Verification Summary

As documented in test report TC-IR-001, the register achieves 100% statement and branch coverage[cite: 1335]. Verified features include:

- **Reset Accuracy:** Confirmed output is $0x0000$ after reset[cite: 1112, 1254].
- **Latching Integrity:** Validated correct latching of hexadecimal patterns `0xAAAA` and `0x5555`[cite: 1269, 1275].
- **Asynchronous Timing:** Verified that reset triggers immediately without waiting for a clock edge[cite: 1302, 1359].

8 CONTROL UNIT (CU)

The Control Unit is the central Finite State Machine (FSM) driving the processor's datapath. It generates specific control signals for the ALU, Register File, and RAM multiplexers.

8.1 Implementation Schematic

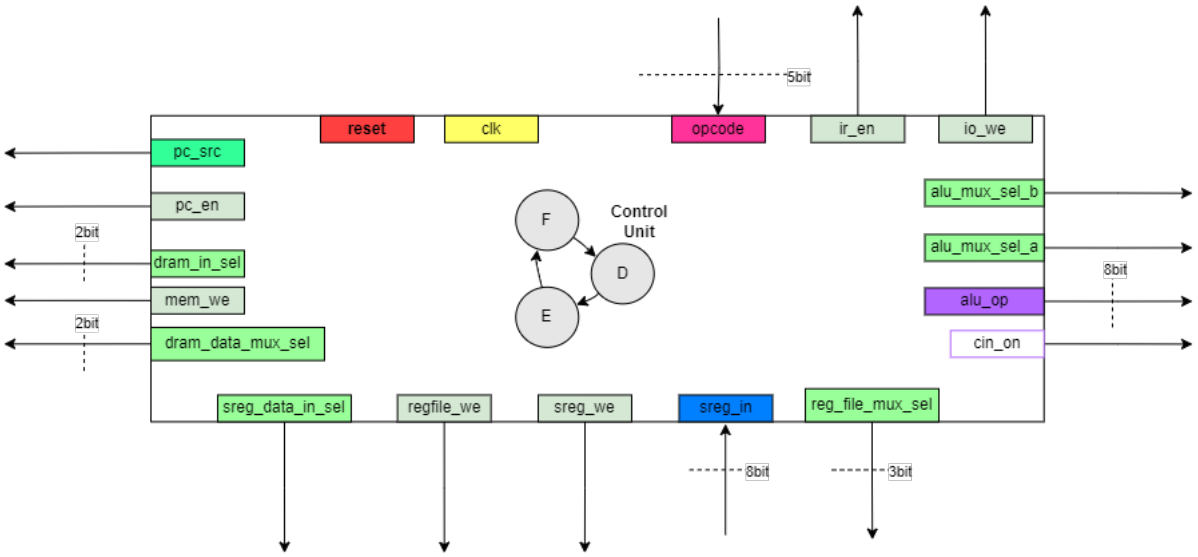

Figure 7: Control Unit

8.2 Finite State Machine Operation

The Control Unit operates as a synchronous Finite State Machine (FSM) with three distinct states. The FSM cycles linearly through these states to process every instruction, ensuring signal stability and correct pipeline management.

8.2.1 FSM States

The state transition logic acts as the sequencer for the processor. The states are defined as follows:

1. FETCH State

Function: Retrieves the next instruction from the Program Memory (ROM) and updates the Program Counter.

- `ir_en` ← **'1'**: Latches the instruction word from the `prom_data` bus into the Instruction Register (IR).
- `pc_en` ← **'1'**: Enables the Program Counter to update.
- `pc_src` ← **"00"**: Selects "Fetch Mode" ($PC \leftarrow PC+1$) to point to the next sequential instruction.

2. DECODE State

Function: A stabilization state that allows the combinational Instruction Decoder to slice the instruction word and the Register File to output data.

- **No Write Enables:** Generally, no write signals are active in this state to prevent race conditions.
- **Activity:** The Decoder splits the instruction into `opcode`, `RD`, `RS`, and `Immediate` values. The Register File asynchronously reads source registers based on these decoded addresses.

3. EXECUTE State

Function: Performs the actual operation (ALU calculation, Memory access, or Branching) and commits results. The control signals in this state depend entirely on the `opcode`.

- **ALU Operations:** `alu_sel` outputs the specific opcode. `regfile_we` is asserted to save the result. `sreg_we` is asserted to update status flags.
- **Memory Store:** `mem_we` is asserted to write data to RAM.
- **Branch/Jump:** The `pc_src` signal is updated based on logic:
  - `"01"`: Relative Branch (e.g., `RJMP`).
  - `"11"`: Absolute Jump (e.g., `JMP`).

8.2.2 Operational Logic

The Control Unit acts as the coordinator for the datapath components:

1. **Input Analysis:** The CU continuously monitors the `opcode` (from the Decoder) and `sreg_data` (flags like Zero or Carry from the Status Register).
2. **State Transitions:** The FSM transitions linearly on every clock cycle:
   FETCH $\rightarrow$ **DECODE** $\rightarrow$ **EXECUTE** $\rightarrow$ **FETCH**
3. **Signal Generation:** Inside the `EXECUTE` state, a combinational logic block maps the current `opcode` to the specific vector of control signals.

8.2.3 VHDL State Definition

The states are enumerated in the VHDL architecture as follows:

```
type state_type is (FETCH, DECODE, EXECUTE);
signal current_state, next_state : state_type;
```

8.3 Interface Signals

The interface signals are derived from the **PolyFlop8 Block Specification Manual Rev 1.2** and validated against the VHDL entity definition.

Signal Name	Dir	Width	Description
System Inputs			
clk	IN	1-bit	System Clock
rst	IN	1-bit	Active High Reset
Datapath Inputs			
opcode	IN	5-bit	Opcode from Decoder (Bits 15:11)
sreg_data	IN	8-bit	Status flags for conditional branching
Control Outputs			
alu_sel	OUT	4-bit	ALU Operation Opcode
alu_mux_sel_a	OUT	1-bit	ALU Mux A (0=RegFile, 1=PC)
alu_mux_sel_b	OUT	1-bit	ALU Mux B (0=RegFile, 1=Imm)
rf_src_sel	OUT	3-bit	RegFile Write-Back Source Selector
mem_addr_sel	OUT	2-bit	RAM Address Source Selector
mem_data_sel	OUT	2-bit	RAM Data Source Selector
pc_src	OUT	2-bit	PC Next Address Mode
pc_en	OUT	1-bit	Program Counter Enable (Stall Control)
ir_en	OUT	1-bit	Instruction Register Enable
mem_we	OUT	1-bit	Data RAM Write Enable
regfile_we	OUT	1-bit	Register File Write Enable
sreg_we	OUT	1-bit	Status Register Write Enable
io_we	OUT	1-bit	I/O Write Enable

Table 6: Control Unit Entity Port Definition

9 STATUS REGISTER (STATUSREG)

The Status Register (SREG) serves as the processor's state storage, capturing arithmetic flags from the ALU and maintaining system control bits. It is implemented as an 8-bit register with asynchronous reset and synchronous write capabilities.

9.1 Implementation Schematic

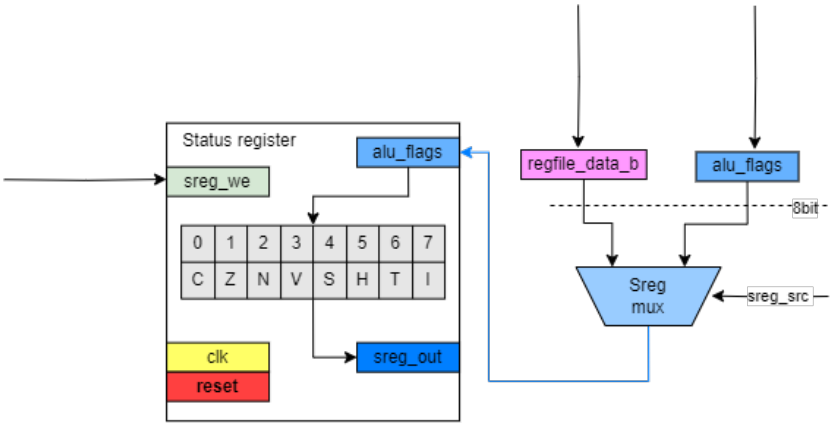

Figure 8: Status Register Logic and ALU Interface

9.2 Internal Flag Mapping

The register bits are mapped to specific ALU flags or preserved control bits. Note the swapping of indices for Carry and Zero flags relative to the input vector, as defined in the VHDL architecture.

Bit	Flag	Source (ALU Mode)	Description
0	**C**	`alu_flags(1)`	Carry Flag
1	**Z**	`alu_flags(0)`	Zero Flag
2	**N**	`alu_flags(2)`	Negative Flag
3	**V**	`alu_flags(3)`	Overflow Flag (2's Complement)
4	**S**	$N \oplus V$	Sign Flag (Unique calculation)
5	**H**	`alu_flags(4)`	Half-Carry (BCD operations)
6	**T**	*Preserved*	Bit Copy / Transfer Storage
7	**I**	*Preserved*	Global Interrupt Enable

Table 7: Status Register Bit Assignments

9.3 Functional Description

The register operates in two distinct modes controlled by the `sreg_src` multiplexer signal:

- **ALU Update Mode (`sreg_src = '0'`):** Active during arithmetic and logical instructions.
  - Flags **C, Z, N, V, H** are updated directly from the `alu_flags` input.
  - The **S (Sign)** flag is automatically calculated as $S = N \oplus V$ to support signed conditional branches (Verified in test case `TC-SREG-001-03`).
  - The **T** and **I** flags are explicitly preserved (fed back from current state) to prevent corruption during arithmetic operations (Verified in test case `TC-SREG-001-04`).
- **Restore Mode (`sreg_src = '1'`):** Used for full-state updates, such as returning from an interrupt or popping the status from the stack. The entire 8-bit byte from `regfile_data_b` is written to the register, overwriting all flags including I and T.

9.4 Reset and Timing

The Status Register utilizes an **asynchronous reset** (`rst`). Upon assertion, the register is immediately cleared to `0x00`, ensuring interrupts are disabled and flags are cleared at system startup. All write operations are synchronous to the rising edge of `clk` when `sreg_we` is high.

10 DATA RAM

The Data RAM entity integrates a 256-byte storage array with input multiplexers for address and data buses. It serves as the primary data storage for the processor, supporting both direct and indirect addressing modes as well as stack operations for subroutine calls.

10.1 Implementation Schematic

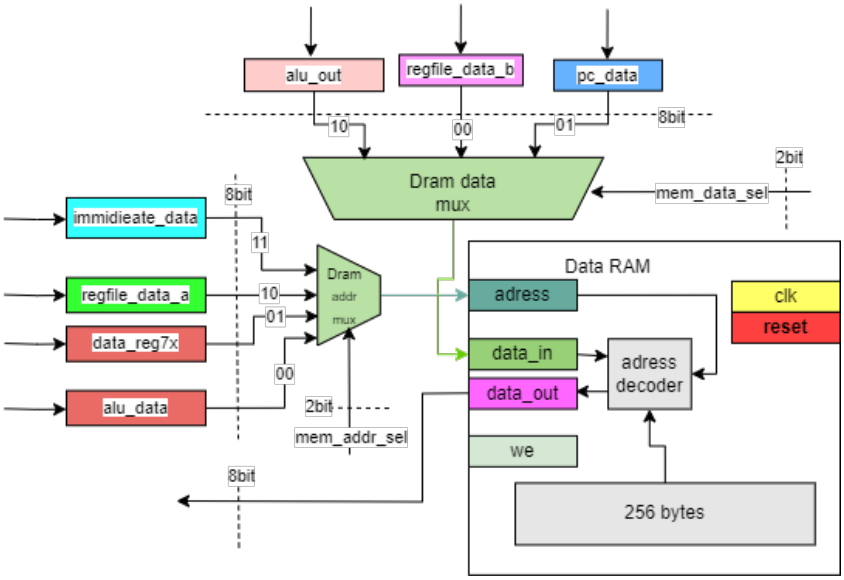

Figure 9: Data RAM Entity Block Diagram

10.2 Address Multiplexer (`mem_addr_sel`)

The address multiplexer selects the source pointer used to access memory based on the instruction type.

Sel	Signal Source	Addressing Mode / Instruction
00	`alu_out`	Base + Offset / Computed Address
01	`data_reg7x`	Indirect via X-Pointer (`LD Rd, X`, `ST X, Rs`)
10	`regfile_data_a`	Indirect via Register A
11	`rs_imm_data`	Direct Addressing (`LDS`, `STS`)

Table 8: Data RAM Address Multiplexer Configuration

10.3 Data Input Multiplexer (mem_data_sel)

This multiplexer selects the data to be written to the RAM array when the write enable signal (mem_we) is active.

Sel	Signal Source	Operation
00	regfile_data_b	Store Register (ST, STS)
01	pc_data	Store Return Address (RCALL)
10	alu_out	Store Calculation Result (ALU Write)
11	0x00	Unused / Default Zero Drive

Table 9: Data RAM Data Input Multiplexer Configuration

10.4 Timing and Control

The Data RAM operates with mixed timing characteristics to ensure signal stability and correct latching:

- **Read Operation (Asynchronous):** Data appears on ram_data_out immediately after the address input stabilizes, subject to internal propagation delays.
- **Write Operation (Synchronous):** Data is written to the memory array on the rising edge of the system clock (clk) when the write enable signal (mem_we) is high.

11 I/O UNIT

The I/O Unit handles communication with external peripherals using **Port-Mapped (Isolated) I/O**. Unlike Memory Mapped I/O, this unit uses a dedicated address space separate from the Data RAM, allowing the use of the full 8-bit immediate field from the Instruction Register as a port address.

11.1 Implementation Schematic

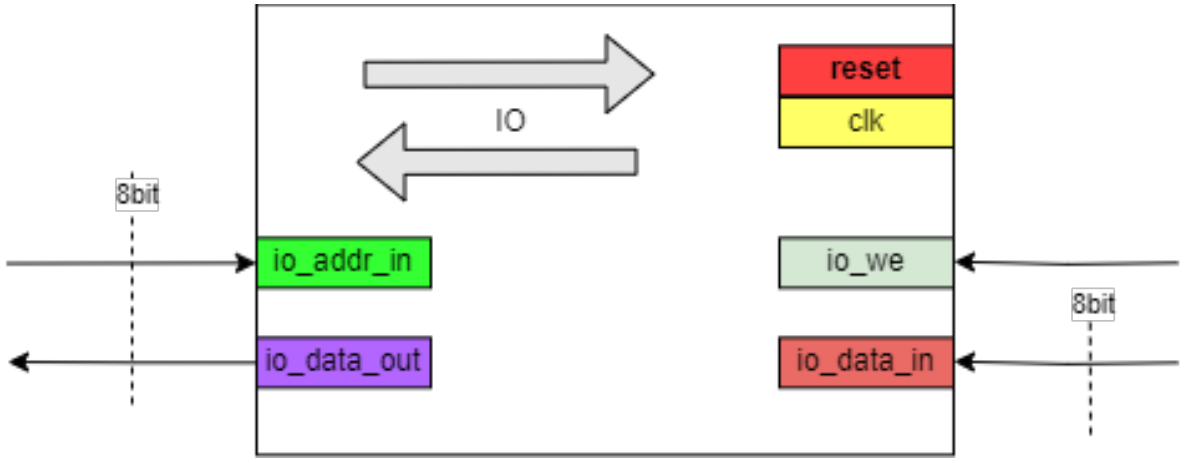

Figure 10: I/O Unit

11.2 Interface Signals

Signal Name	Dir	Width	Description
`clk`, `rst`	IN	1-bit	System Clock and Active High Reset
`io_we`	IN	1-bit	Write Enable (Strobe)
`io_addr_in`	IN	8-bit	Port Address (Derived from IR Immediate)
`io_data_out`	IN	8-bit	Data from CPU (to be written to Output)
`io_data_in`	OUT	8-bit	Data to CPU (read from Input)
`port_out_a`	OUT	8-bit	Physical Output Pins (e.g., LEDs)
`pin_in_b`	IN	8-bit	Physical Input Pins (e.g., Switches)

Table 10: I/O Unit Entity Port Definition

11.3 I/O Address Map

The I/O unit decodes the `io_addr_in` bus to select the active peripheral. The address space is 8-bit (0x00 to 0xFF).

Address (Hex)	Target Register	Function
0x01	reg_port_a	**Write**: Latch data to Port A Output
0x02	pin_in_b	**Read**: Sample data from Port B Input
Others	N/A	Reserved

11.4 Functional Logic

- **Output Operation (OUT):** When io_we is high and io_addr_in matches 0x01, the data from the internal io_data_out bus is latched into reg_port_a, driving port_out_a.
- **Input Operation (IN):** The unit acts as a combinational mux for reads. When io_addr_in matches 0x02, the state of physical pins pin_in_b is driven onto the io_data_in bus.
- **Integration Note:** The io_data_in bus connects to the Register File Input Mux (Select 011), while io_addr_in connects directly to the Instruction Register's immediate field (IR[7:0]).